

PYNQ简介

目录

- PYNQ能简化ZYNQ开发吗
- 为什么还要用PYNQ呢?
- 不会python能用PYNQ吗?
- PYNQ学习资源

Can PYNQ simplify ZYNQ development?

The main goal of PYNQ is to use **Python** to develop **Zynq**, so that designers can quickly experience zynq-based embedded development, according to Xilinx's official statement: let people who do not understand FPGAs or even C language develop ZYNQ. Is this really the case? Let's take a look at what is the core part of PYNQ?

Overlays, this is difficult to express clearly in Chinese, the essence is the result of FPGA compilation, it is a bit stream file, PYNQ can dynamically load and unload these bit streams, so that the functions of the FPGA side can be dynamically changed according to the needs of ARM software. So, if you don't know how to develop FPGAs, how do these bit stream files come from? If you are in a large company, you can find an FPGA developer to develop, if it is a company of average scale, then the developer should honestly learn FPGA first, about a year of unremitting practice.

PYNQ uses python as the development language, so far, C or C++ is the most commonly used embedded programming language, because embedded often deals with the underlying hardware, although python can improve development efficiency, but there is still someone to develop libraries for Python, these underlying libraries are basically C or C++.

PYNQ is browser-based and uses a technology called Jupyter notebook, which is not unique to ZYNQ, and ordinary PCs can also do it.

To sum up:

[Skip to main content](#)

- FPGAs still need to be developed
- Embedded developers still need to provide the underlying interface to Python
- ZYNQ development cannot be simplified

Why use PYNQ?

- Very familiar with Python, the previous algorithms were written in Python, and now they need to be ported
- It really doesn't work in C or C++, and it doesn't build a cross-compiled environment
- Xilinx provides some Python libraries for the underlying hardware, such as register reading and writing, DMA use, with these basic operations, it can solve most of the embedded programs that do not require high efficiency, and it is still very convenient for debugging
- PYNQ is the icing on the cake, and it is a very suitable tool for people who have a Python foundation but do not want to systematically learn embedded development while ZYNQ in school

Can't python use PYNQ?

If you don't know python, this is really not good!

PYNQ Learning Resources

Official website: <http://www.pynq.io>

Github:  [Xilinx/PYNQ](#)

PYNQ快速上手

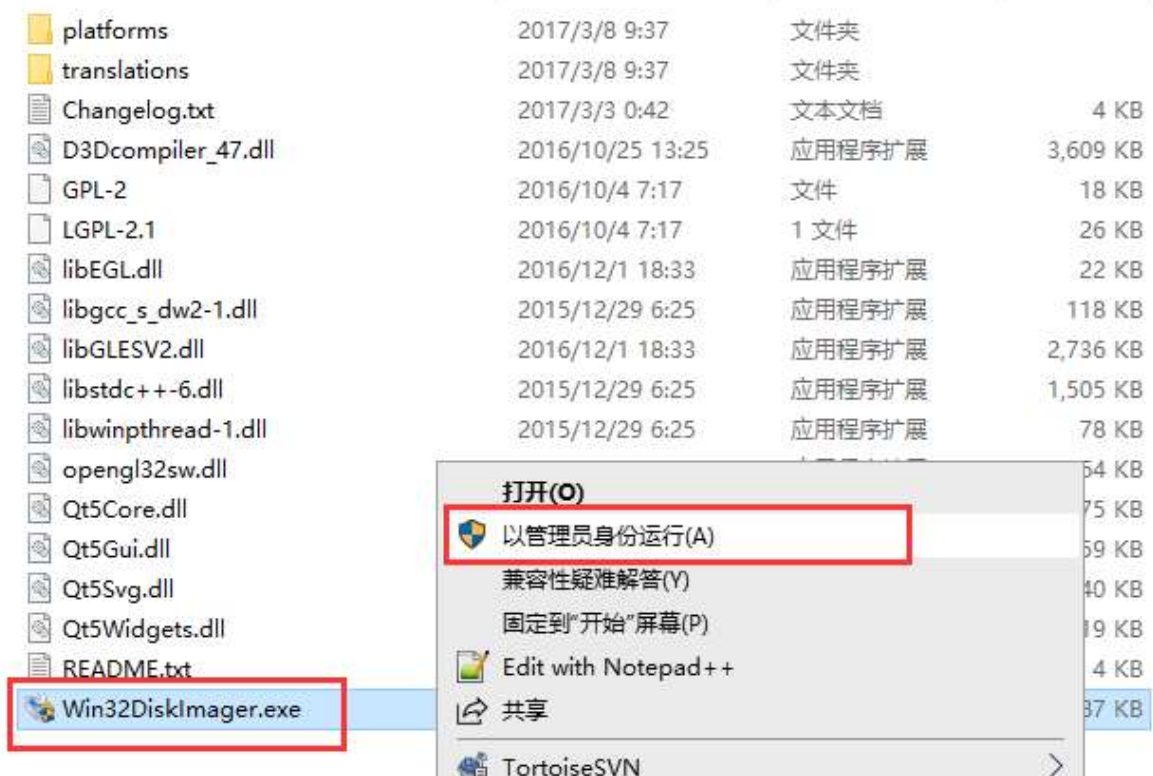
目录

- PYNQ镜像
- 运行PYNQ
- samba文件共享
- 常见问题

PYNQ Mirror

AX7010/AX7020 factory program does not support PYNQ, you need to re-program the image to the SD card, the burning method is as follows (for how to make a PYNQ image, you can refer to the relevant tutorial on the official website, this tutorial does not provide image production method)

1. Unzip the image compressed package ax7010-2.4.zip or ax7020-2.4.zip to get a file with the extension img
2. SD card (at least 8G, all original content is lost after burning) is inserted into the USB port of the computer with a card reader, ready to be burned
3. Download the Win32DiskImager image burner tool, which is used for a wide range of purposes, such as Raspberry Pi burning images, and then running them with administrator privileges



4. Select the image file, then select the device, pay attention to the drive letter where the SD card is located, and then click Write

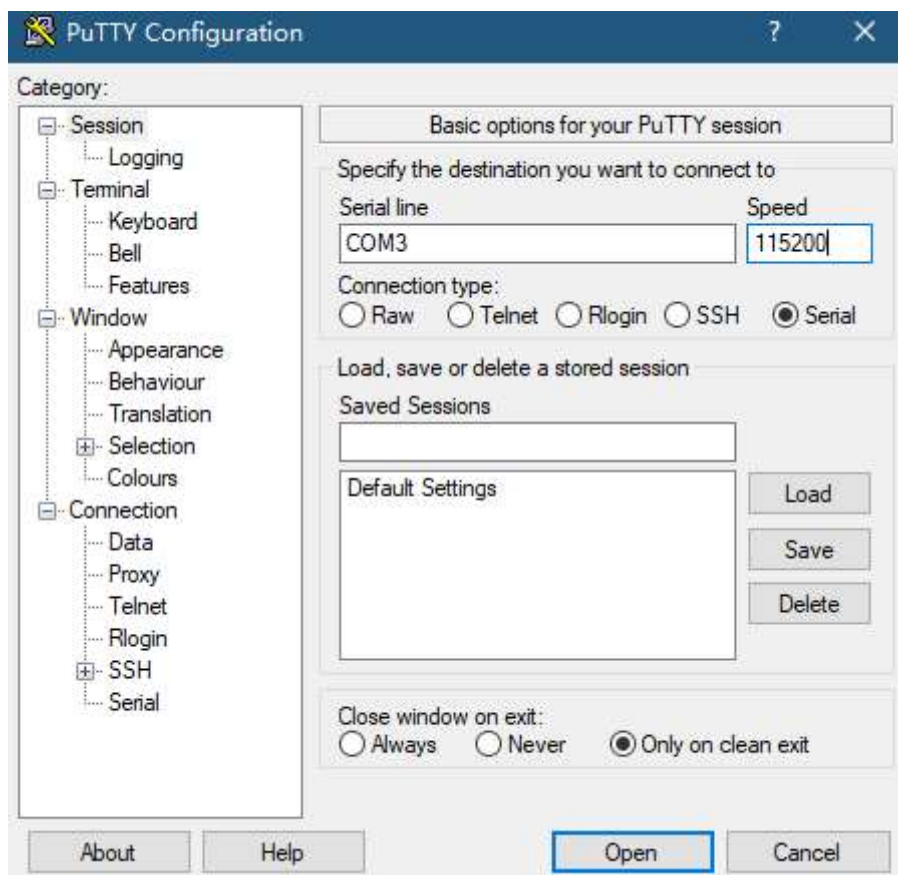


5. After the writing is completed, the SD card is divided into 2 partitions, the first partition is FAT32, which can be seen under Windows, and the second partition is EXT4, which is not visible under Windows.

名称	修改日期	类型	大小
BOOT.BIN	2019/7/31 3:15	BIN 文件	1,517 KB
image.ub	2019/7/31 3:15	UB 文件	4,382 KB

Run PYNQ

1. Insert the SD card back into the board and confirm that the boot mode of the board is SD card mode
2. Connect the network port of the development board to the router (preferably support DHCP), if the router supports DHCP, the development board will automatically obtain the IP address after powering on, and can be accessed with the host name pynq, if it does not support DHCP, the development board will start the static IP address 192.168.2.99
3. Connect the UART port of the development board to the USB port of the computer
4. Use Putty as a serial port debugging tool, refer to the basic tutorial for how to use it



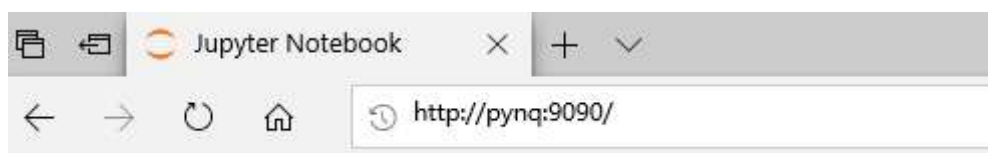
1. Powering on, it can be seen from the print information that the file system is Ubuntu 18.04

```
COM3 - PuTTY
systemd[1]: System time before build time, advancing clock.
systemd[1]: Failed to insert module 'autofs4': No such file or directory
systemd[1]: systemd 237 running in system mode. (+PAM +AUDIT +SELINUX +IMA +APPARMOR +SMACK +SYSVINIT +UTMP +LIBCRYPTSETUP +GCRYPT +GNUTLS +ACL +XZ +LZ4 +SECCOMP +BLKID +ELFUTILS +KMOD -IDN2 +IDN -PCRE2 default-hierarchy=hybrid)
systemd[1]: Detected architecture arm.

Welcome to PynqLinux, based on Ubuntu 18.04!

systemd[1]: Set hostname to <pynq>.
systemd[1]: Created slice System Slice.
[ OK ] Created slice System Slice.
systemd[1]: Listening on udev Kernel Socket.
[ OK ] Listening on udev Kernel Socket.
systemd[1]: Listening on udev Control Socket.
[ OK ] Listening on udev Control Socket.
systemd[1]: Created slice system-serial\x2dgetty.slice.
[ OK ] Created slice system-serial\x2dgetty.slice.
systemd[1]: Created slice User and Session Slice.
[ OK ] Created slice User and Session Slice.
systemd[1]: Reached target Slices.
[ OK ] Reached target Slices.
systemd[1]: Reached target System Time Synchronized.
[ OK ] Reached target System Time Synchronized.
```

6. If the router supports DHCP, enter the <http://pynq:9090> in the browser, if the router does not support DHCP, first modify the computer IP to the 192.168.2.x network segment, and then enter the <http://192.168.2.99:9090> in the browser



7. Enter the login password: xilinx, click "Log in"



8. A file manager-like interface can be seen

Select items to perform actions on them.

☐ 0 

- ☐  base
- ☐  common
- ☐  getting_started
- ☐  resizer
- ☐  tutorial_1
- ☐  tutorial_2
- ☐  Welcome to Pynq.ipynb

9. Find an easy-to-understand example and run it, in the common folder, click "zynq_clocks.ipynb"

Select items to perform actions on them.

☐ 0  / common

-  ..
- ☐  data
- ☐  overlay_download.ipynb
- ☐  python_random.ipynb
- ☐  python_retrieving_shell.ipynb
- ☐  usb_webcam.ipynb
- ☐  wifi.ipynb
- ☐  zynq_clocks.ipynb

10. After opening, there will be a status prompt "Kernel starting, please wait...", at this time you cannot click "Run"

PS Clock Control

This notebook demonstrates how to use `Clocks` class to control the PL clocks.

By default, there are at most 4 PL clocks enabled in the system. They all can be reprogrammed to valid clock rates.

Whenever the overlay is downloaded, the required clocks will also be configured.

References:

https://www.xilinx.com/support/documentation/user_guides/ug585-Zynq-7000-TRM.pdf

Show All Clocks

The following example show all the current clock rates on the board.

```
In [1]: from pyng import Clocks
from pyng import PL, Overlay

print(f'CPU: {Clocks.cpu_mhz:.6f}MHz')
print(f'FCLK0: {Clocks.fclk0_mhz:.6f}MHz')
print(f'FCLK1: {Clocks.fclk1_mhz:.6f}MHz')
print(f'FCLK2: {Clocks.fclk2_mhz:.6f}MHz')
print(f'FCLK3: {Clocks.fclk3_mhz:.6f}MHz')
```

CPU: 650.000000MHz
FCLK0: 100.000000MHz
FCLK1: 142.857143MHz

11. When the state changes to "Kernel idle", you can click "Run"

PS Clock Control

This notebook demonstrates how to use `Clocks` class to control the PL clocks.

By default, there are at most 4 PL clocks enabled in the system. They all can be reprogrammed to valid clock rates.

Whenever the overlay is downloaded, the required clocks will also be configured.

References:

12. With the mouse, you can select the running unit called "cell", there are 4 types of running units, commonly used classes are Markdown and Code, as the name suggests, Code is code, Markdown is text, equivalent to comments, although "cell" can be run, but only those marked as "Code" are meaningful

13. Generally, we start running from the first cell, and after we run the first piece of code, the print result will be given under the cell, and the result shows that the CPU clock is 666.666666Mhz. **Special note: wait for the state to change to "Kernel idle" after each click on Run before running the next cell.**

Show All Clocks

The following example show all the current clock rates on the board.

```
In [1]: from pynq import Clocks
        from pynq import PL, Overlay

        print(f'CPU: {Clocks.cpu_mhz:.6f}MHz')
        print(f'FCLK0: {Clocks.fclk0_mhz:.6f}MHz')
        print(f'FCLK1: {Clocks.fclk1_mhz:.6f}MHz')
        print(f'FCLK2: {Clocks.fclk2_mhz:.6f}MHz')
        print(f'FCLK3: {Clocks.fclk3_mhz:.6f}MHz')
```

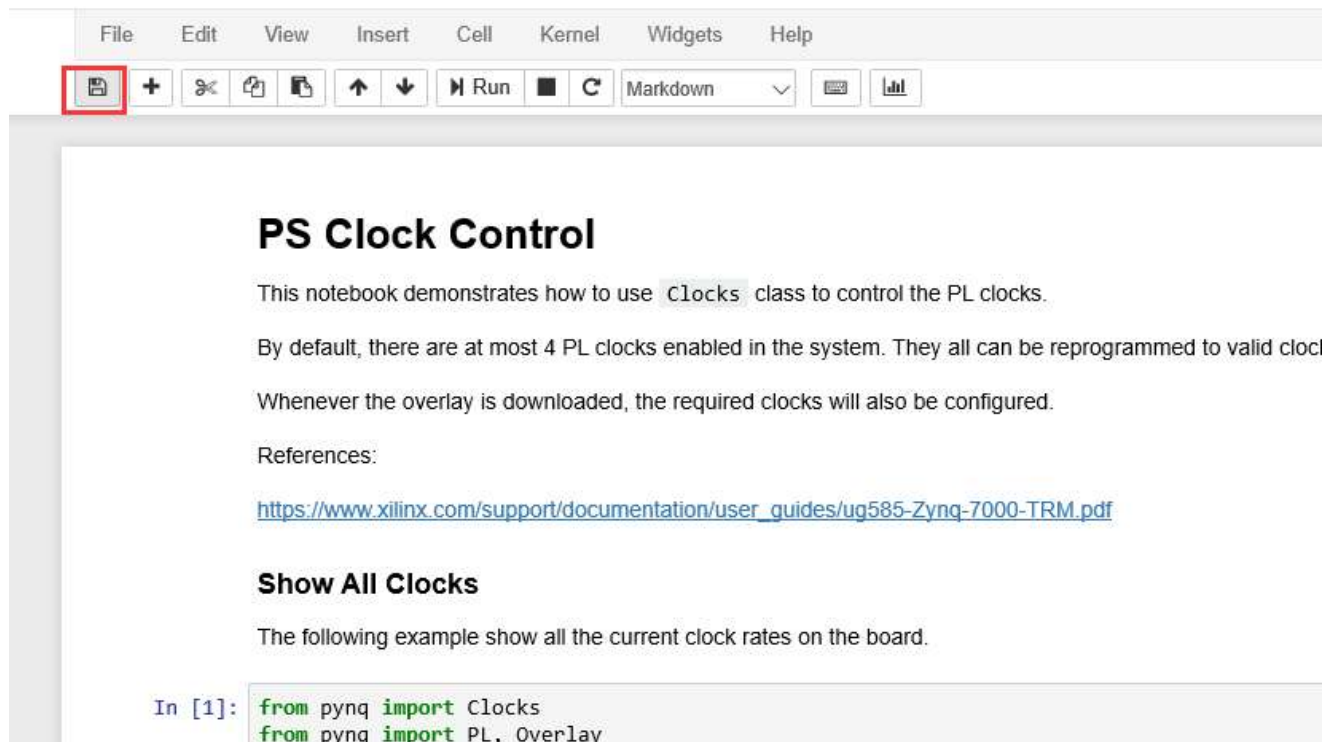
CPU: 666.666660MHz
FCLK0: 99.999999MHz
FCLK1: 142.857141MHz
FCLK2: 199.999998MHz
FCLK3: 166.666665MHz

Set Clock Rates

The easiest way is to set the attributes directly. Random clock rates are used in the following examples; the clock manager will set the clock rates with best effort.

If the desired frequency and the closest possible clock rate differs more than 1%, a warning will be raised.

14. After the run ends, you can click Save, and the current program and status will be saved



The screenshot shows the JupyterLab interface. The top toolbar contains several icons, and the 'Save' icon (a floppy disk) is highlighted with a red square. Below the toolbar, the main content area displays the notebook's text, which includes the title 'PS Clock Control', an introduction to the 'Clocks' class, and the 'Show All Clocks' section with the same code and output as shown in the previous blocks.

PS Clock Control

This notebook demonstrates how to use `Clocks` class to control the PL clocks.

By default, there are at most 4 PL clocks enabled in the system. They all can be reprogrammed to valid clock rates.

Whenever the overlay is downloaded, the required clocks will also be configured.

References:

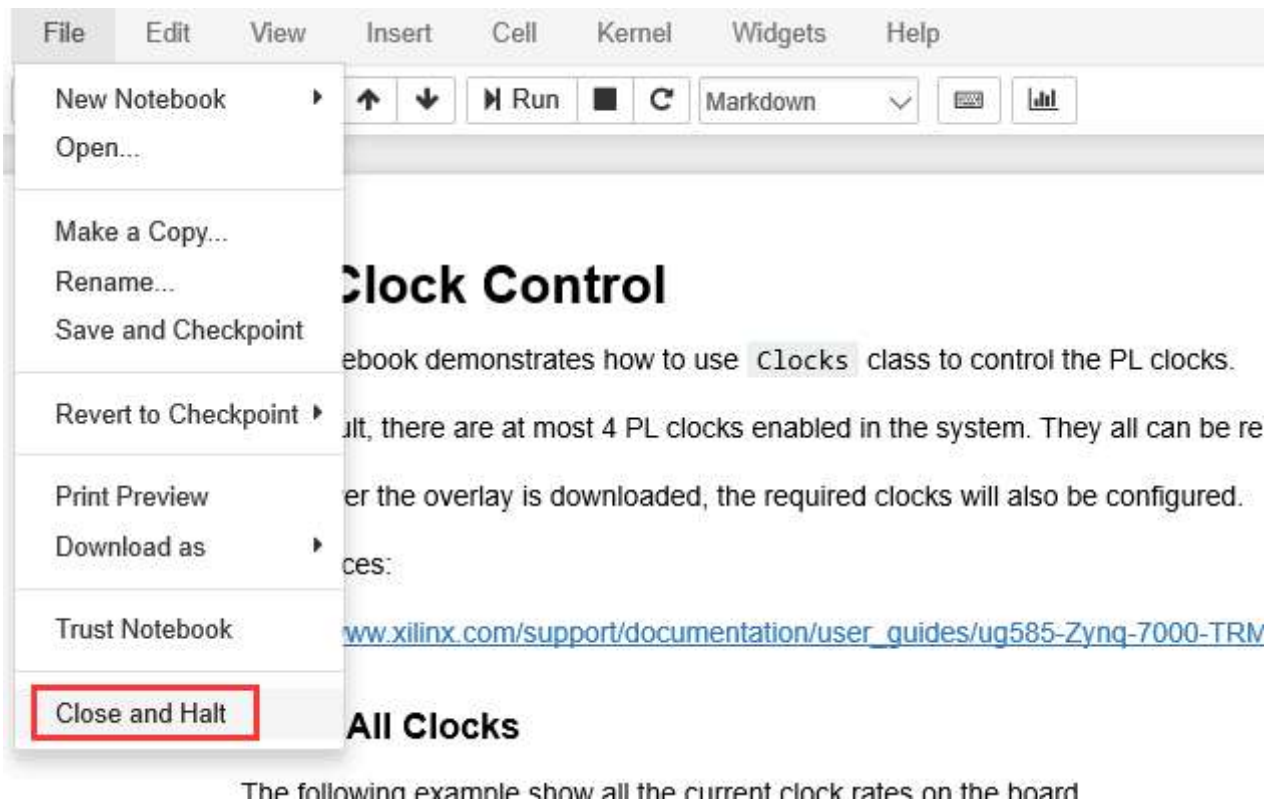
https://www.xilinx.com/support/documentation/user_guides/ug585-Zynq-7000-TRM.pdf

Show All Clocks

The following example show all the current clock rates on the board.

```
In [1]: from pynq import Clocks
        from pynq import PL, Overlay
```

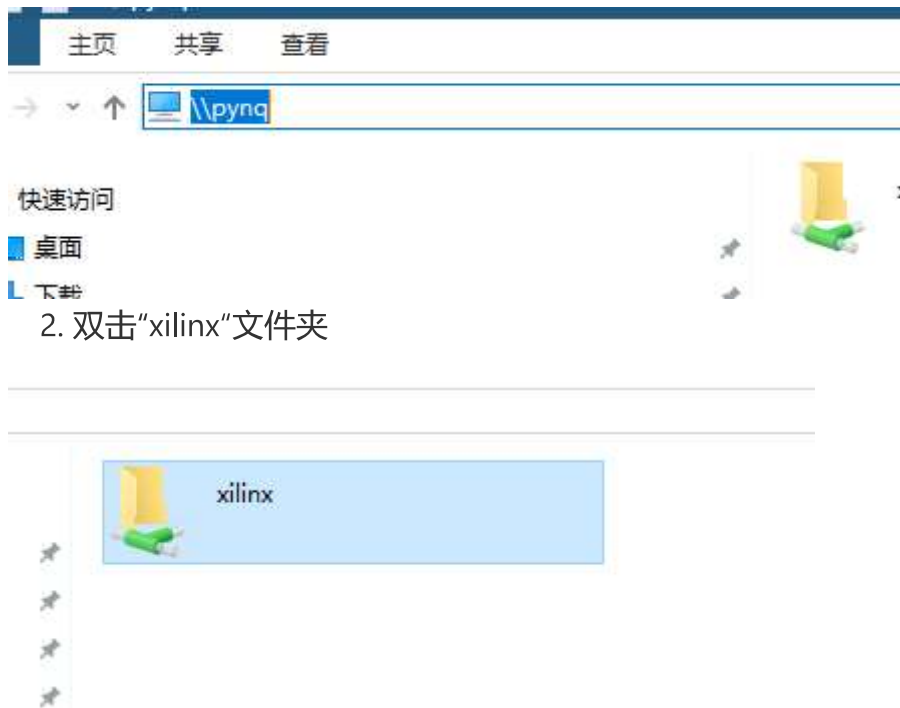
15. Finally, you can click File → Close and Halt to exit the program



samba文件共享

PYNQ提供了samba共享文件夹，方便上传和下载文件，使用方法如下：

1. 路由器支持DHCP时在资源管理器里输入\\pynq，不支持的输入\\192.168.2.99



2. 双击"xilinx"文件夹
3. 用户名和密码都输入"xilinx"



4. You can see the folders and files on the board

名称	修改日期	类型
jupyter_notebooks	2019/8/13 9:19	文件夹
pynq	2019/8/13 11:34	文件夹
REVISION	2019/7/31 11:15	文件

frequently asked questions

<http://pynq:9090> not accessible using it

Some systems cannot be accessed through hostname, this use we need to use the IP address to access directly, enter the command `ifconfig` in the serial terminal, the ip address of `eth0` is DHCP assigned to the ip address, if there is no address, it means that DHCP allocation failed. Enter <http://192.168.1.212:9090> in your browser instead of <http://pynq:9090>

```
xilinx@pyng:~$ ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.1.212 netmask 255.255.255.0 broadcast 192.168.1.255
    inet6 fe80::2c85:45ff:fe36:320e prefixlen 64 scopeid 0x20<link>
    ether 2e:85:45:36:32:0e txqueuelen 1000 (Ethernet)
    RX packets 16397 bytes 1772321 (1.7 MB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 9839 bytes 10055997 (10.0 MB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
    device interrupt 29 base 0xb000

eth0:1: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.2.99 netmask 255.255.255.0 broadcast 192.168.2.255
    ether 2e:85:45:36:32:0e txqueuelen 1000 (Ethernet)
    device interrupt 29 base 0xb000

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 1317 bytes 163650 (163.6 KB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 1317 bytes 163650 (163.6 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
```

USB摄像头边沿检测和人脸识别

目录

- 边沿检测
- 人脸检测

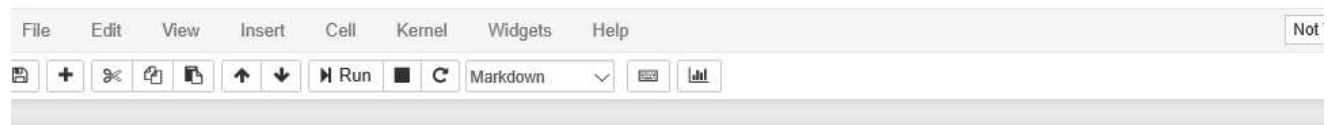
In the pynq image, a lot of built-in routines are given, the more representative ones are edge detection and face recognition, unfortunately, these two routines do not use FPGA for acceleration, they are purely software calculated, but the HDMI output of the FPGA is used to display the results. These routines are officially provided by Xilinx, and Xinyi Electronics has done some localization work.

Edge detection

As in the previous chapter, log in to Jupyter Notebooks and find the notebook `opencv_filters_webcam.ipynb` in the directory `base/video`.

Before running this routine, connect the USB camera (drive-free USB camera).

This tutorial looks particularly pale and weak under the powerful annotation function of Notebooks, and each step can have rich hints. The Python language itself is not the focus of this tutorial, if you are not familiar with the Python language, you can strengthen the practice on the computer first.



OpenCV Filters Webcam

In this notebook, several filters will be applied to webcam images.

Those input sources and applied filters will then be displayed either directly in the notebook or on HDMI output.

To run all cells in this notebook a webcam and HDMI output monitor are required.

1. Start HDMI output

Step 1: Load the overlay请耐心等待后加载成功后再执行下一步

```
In [1]: from pynq.overlays.base import BaseOverlay
        from pynq.lib.video import *
        base = BaseOverlay("base.bit")
```

Step 2: Initialize HDMI I/O

```
In [2]: # monitor configuration: 640*480 @ 60Hz
        Mode = VideoMode(640,480,24)
        hdmi_out = base.video.hdmi_out
        hdmi_out.configure(Mode,PIXEL_BGR)
        hdmi_out.start()
```

```
Out[2]: <contextlib._GeneratorContextManager at 0xaffbac50>
```

2. Applying OpenCV filters on Webcam input

Step 1: Specify webcam resolution

```
In [3]: # camera (input) configuration
```

If you use FPGA resources, you must load the bit file of FPGA first.

Through this routine, we can see that it supports commonly used libraries in Python, such as opencv, etc., and if you need to install other libraries, you can install it with the pip command.

Through the experiment, it can be seen that the edge detection results can be displayed directly in the browser

Step 6: Show results

Now use matplotlib to show filtered webcam input inside notebook.

```
: %matplotlib inline
from matplotlib import pyplot as plt
import numpy as np

frame_canny = cv2.Canny(frame_webcam, 100, 110)
plt.figure(1, figsize=(10, 10))
frame_vga = np.zeros((480,640,3)).astype(np.uint8)
frame_vga[:, :, 0] = frame_canny
frame_vga[:, :, 1] = frame_canny
frame_vga[:, :, 2] = frame_canny
plt.imshow(frame_vga)
plt.show()
```



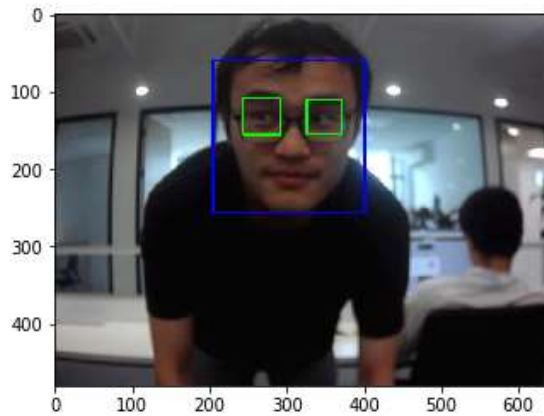
It should be noted that the program involving the FPGA must be fully run every time, and if the program is halfway out of the program, it may lead to abnormal operation next time, and the development board needs to be powered off and restarted.

Face detection

The face detection routine is `opencv_face_detect_webcam.ipynb`. The face detection program mainly detects whether there are faces in the image, and then frames the face and eyes, which is directly called `opencv` for detection

Step 7: Now use matplotlib to show image inside notebook

```
In [9]: # Output OpenCV results via matplotlib
%matplotlib inline
from matplotlib import pyplot as plt
import numpy as np
plt.imshow(np_frame[:, :, [2, 1, 0]])
plt.show()
```



Step 8: Release camera and HDMI

```
In [10]: videoIn.release()
hdmi_out.stop()
del hdmi_out
```

基于寄存器操作自定义Overlays

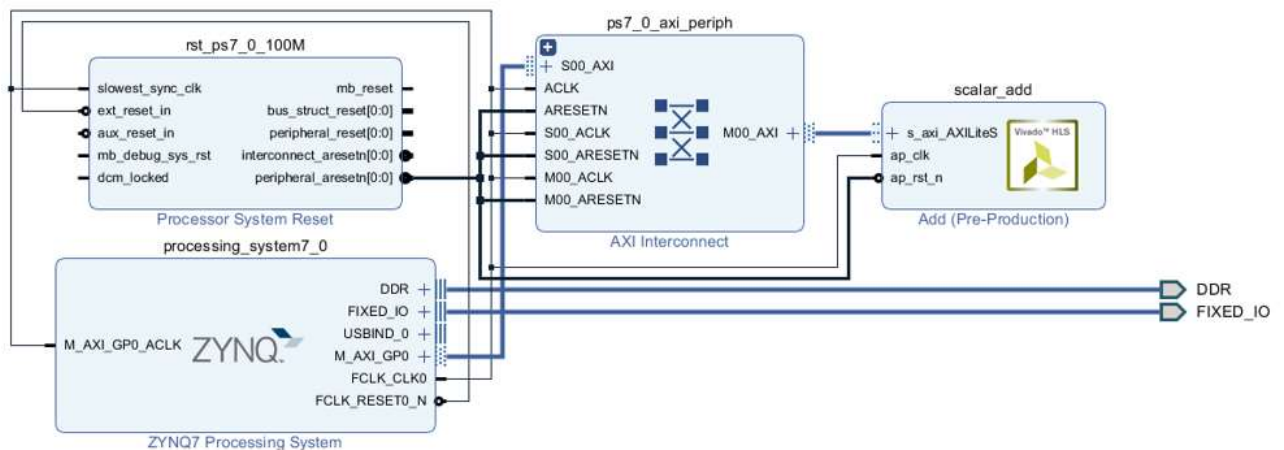
目录

- vivado工程
- 建立botebook

This chapter explains how to import the general vivado project into PYNQ, which is what everyone dreams of learning PYNQ.

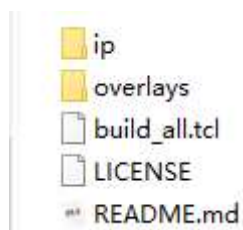
Vivado Project

As shown in the figure below, the core module in the vivado project is a custom IP, written in HLS, the main function is to complete $c = a + b$, as mentioned earlier, xilinx only provides python registers, dma, vdma underlying package, that is to say, not all vivado projects can be directly imported into PYNQ, only some register read and write, dma operation projects.



HLS and Vivado project establishment is not the focus of this tutorial, how to use HLS and Vivado can refer to other tutorials, here is a brief introduction.

In the directory of the given information overlay_tutorial_17_4 there is a content of the IP, IP is the IP source code written by HLS, and Overlays is the Vivado project and TCL file of Overlays. build_all.tcl script is HLS compilation and Vivado project compilation script.



The given routine has been compiled, and tutorial_1, tutorial_2, tutorial_1.bit, tutorial_1.hwh, tutorial_2, tutorial_2.bit, tutorial_2.hwh are generated, and if you want to compile it again, you need to delete these files first. tutorial_1.bit and tutorial_1.hwh are the two files needed for this tutorial, and tutorial_2.bit and tutorial_2.hwh are the files needed for subsequent tutorials.

A screenshot of a Windows File Explorer window. The address bar shows the path: 新加坡 (D:) > test > PYNQ > ax7010 > overlay_tutorial_17_4 > overlays. The main area displays a list of files and folders. The first two items, 'tutorial_1' and 'tutorial_2', are folders and are highlighted with a red rectangle. The remaining items are files: 'tutorial_1.bit', 'tutorial_1.hwh', 'tutorial_1.tcl', 'tutorial_2.bit', 'tutorial_2.hwh', and 'tutorial_2.tcl'. The table has four columns: '名称' (Name), '修改日期' (Modified Date), '类型' (Type), and '大小' (Size).

名称	修改日期	类型	大小
tutorial_1	2019/9/23 10:40	文件夹	
tutorial_2	2019/9/23 10:40	文件夹	
tutorial_1.bit	2019/8/1 16:36	BIT 文件	2,036 KB
tutorial_1.hwh	2019/8/1 16:30	HWH 文件	139 KB
tutorial_1.tcl	2019/8/1 16:23	TCL 文件	48 KB
tutorial_2.bit	2019/8/1 16:55	BIT 文件	2,036 KB
tutorial_2.hwh	2019/8/1 16:37	HWH 文件	345 KB
tutorial_2.tcl	2019/8/1 16:24	TCL 文件	56 KB

We are already very familiar with the bit file, the hwh file is also overview, here is another explanation, the hwh file is an XML document describing the hardware connection information, through the hwh file can obtain the IP information, register information, interrupt information in the hardware design, PYNQ framework by parsing the hwh file to automatically match the relevant IP driver, automatically obtain the IP address information, etc.

Create a botebook

1. Log in to the PYNQ notebook using your browser
2. Create a new folder with "New-> Folder"

jupyter Logout

Files Running Clusters Nbextensions

Select items to perform actions on them.

Upload New ↕ ↻

☐	0			
☐	base			
☐	common			
☐	getting_started			
☐	resizer			
☐	test			2 个月前
☐	test2			2 个月前
☐	Welcome to Pynq.ipynb			2 个月前

Notebook:
Python 3
Other:
Text File
Folder
Terminal

3. Renamed to "overlay_tutorial"

Files Running Clusters Nbextensions

Rename Move

Up

1

Name ↓

☐	base	
☐	common	
☐	getting_started	
☐	resizer	
☐	test	
☐	test2	
☒	Untitled Folder	
☐	Welcome to Pynq.ipynb	

4. Go to the directory "overlay_tutorial" and click "Upload" to upload the file

jupyter

Files Running Clusters Nbextensions

Select items to perform actions on them.

Upload New ↕ ↻

0

/ overlay_tutorial

Name ↓ Last Modified

.. 几秒前

The notebook list is empty.

5. Select tutorial_1.bit, tutorial_1.hwh to upload, and click "Upload"

Files Running Clusters Nbextensions

Select items to perform actions on them. Upload New ↕

0 / overlay_tutorial Name Last Modified

The notebook list is empty.

☐ tutorial_1.hwh	Upload	Cancel
☐ ..		几秒前
☐ tutorial_1.bit	Upload	Cancel

6. The upload is complete

Files Running Clusters Nbextensions

Select items to perform actions on them. Upload New ↕

0 / overlay_tutorial Name Last Modified

☐ ..		几秒前
☐ tutorial_1.bit		几秒前
☐ tutorial_1.hwh		几秒前

7. Create a new notebook

jupyter Logout

Files Running Clusters Nbextensions

Select items to perform actions on them. Upload New ↕

0 / overlay_tutorial

☐ ..		
☐ tutorial_1.bit		
☐ tutorial_1.hwh		

Notebook:
Python 3
Other:
Text File
Folder
Terminal

8. The notebook editing interface pops up, and the default is a Code type cell.

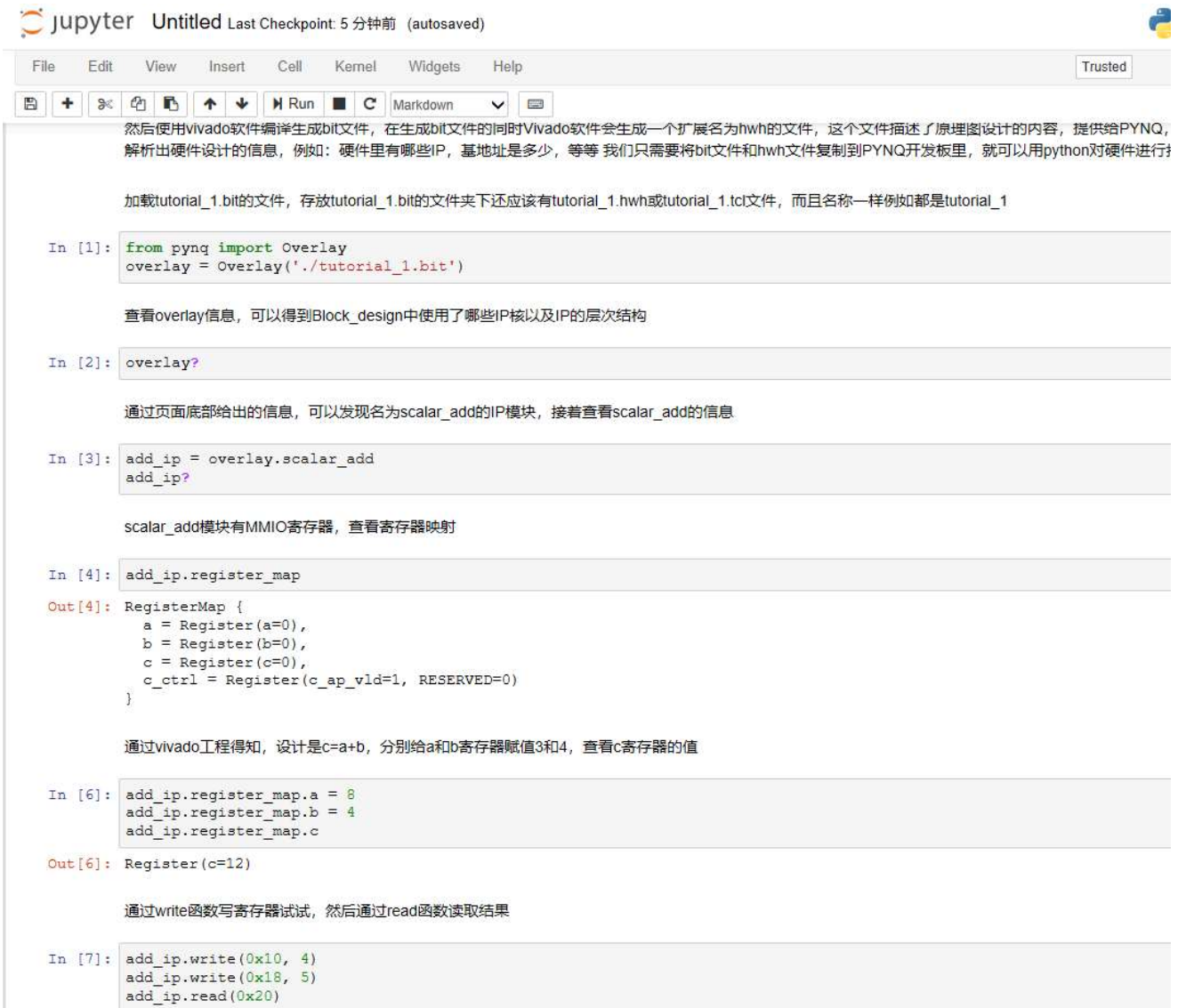
jupyter Untitled (unsaved changes) Python 3 Logout

File Edit View Insert Cell Kernel Widgets Help Trusted Python 3

Code

In []:

9. Start editing a notebook, the editing process is relatively simple, similar to the use of word, the editing process is not the focus of the tutorial



The image shows a Jupyter Notebook interface with the title "Untitled" and a status bar indicating "Last Checkpoint: 5 分钟前 (autosaved)". The interface includes a menu bar (File, Edit, View, Insert, Cell, Kernel, Widgets, Help) and a toolbar with icons for file operations, running, and markdown editing. The notebook content consists of several code cells and text annotations in Chinese.

Text annotations:

- 然后使用vivado软件编译生成bit文件，在生成bit文件的同时Vivado软件会生成一个扩展名为hwh的文件，这个文件描述了原理图设计的内容，提供给PYNQ，解析出硬件设计的信息，例如：硬件里有哪些IP，基地址是多少，等等 我们只需要将bit文件和hwh文件复制到PYNQ开发板里，就可以用python对硬件进行控制
- 加载tutorial_1.bit的文件，存放tutorial_1.bit的文件夹下还应该有tutorial_1.hwh或tutorial_1.tcl文件，而且名称一样例如都是tutorial_1
- 查看overlay信息，可以得到Block_design中使用了哪些IP核以及IP的层次结构
- 通过页面底部给出的信息，可以发现名为scalar_add的IP模块，接着查看scalar_add的信息
- scalar_add模块有MMIO寄存器，查看寄存器映射
- 通过vivado工程得知，设计是c=a+b，分别给a和b寄存器赋值3和4，查看c寄存器的值
- 通过write函数写寄存器试试，然后通过read函数读取结果

Code cells:

```
In [1]: from pynq import Overlay
overlay = Overlay('./tutorial_1.bit')
```

```
In [2]: overlay?
```

```
In [3]: add_ip = overlay.scalar_add
add_ip?
```

```
In [4]: add_ip.register_map
```

```
Out[4]: RegisterMap {
  a = Register(a=0),
  b = Register(b=0),
  c = Register(c=0),
  c_ctrl = Register(c_ap_vld=1, RESERVED=0)
}
```

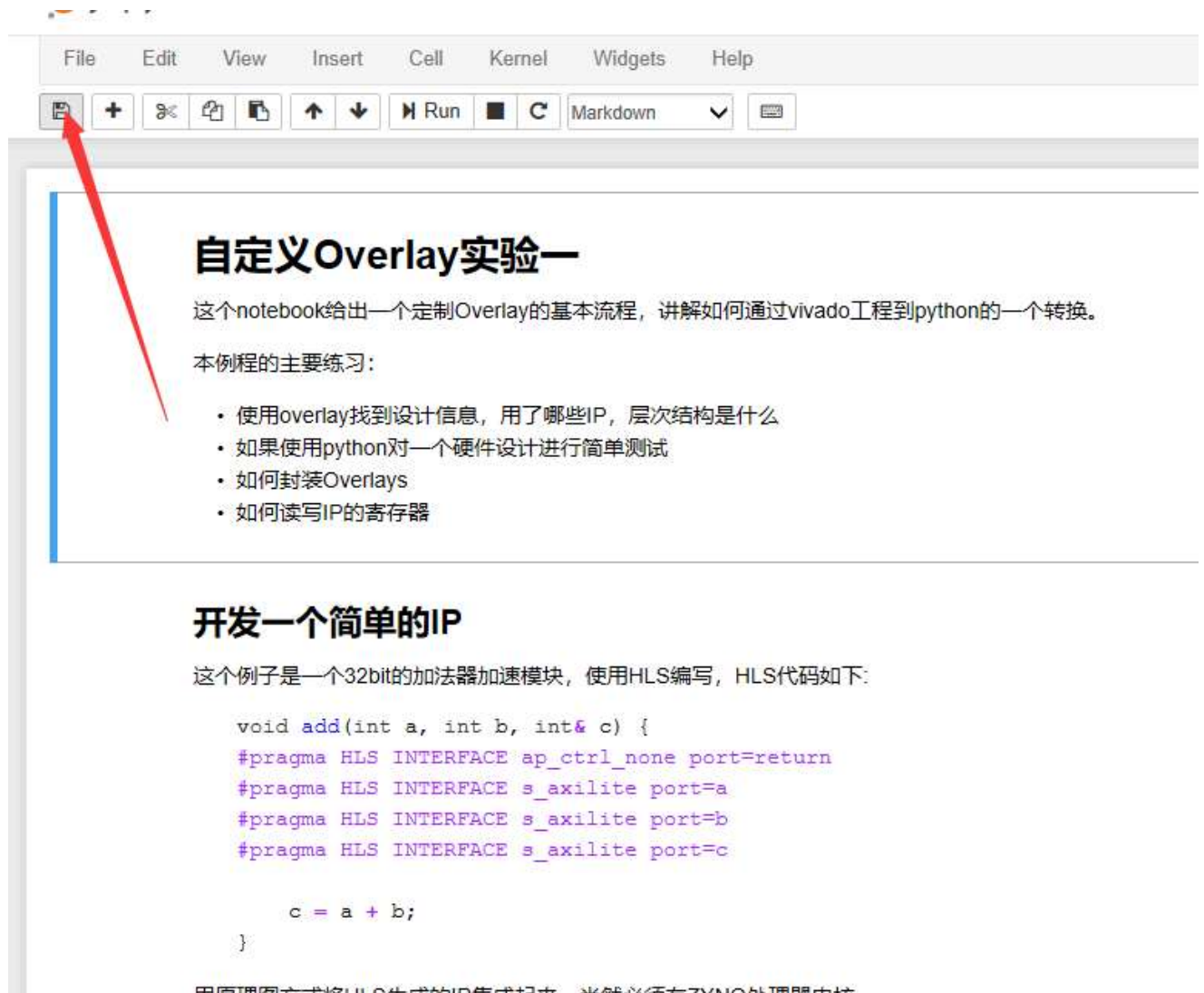
```
In [6]: add_ip.register_map.a = 8
add_ip.register_map.b = 4
add_ip.register_map.c
```

```
Out[6]: Register(c=12)
```

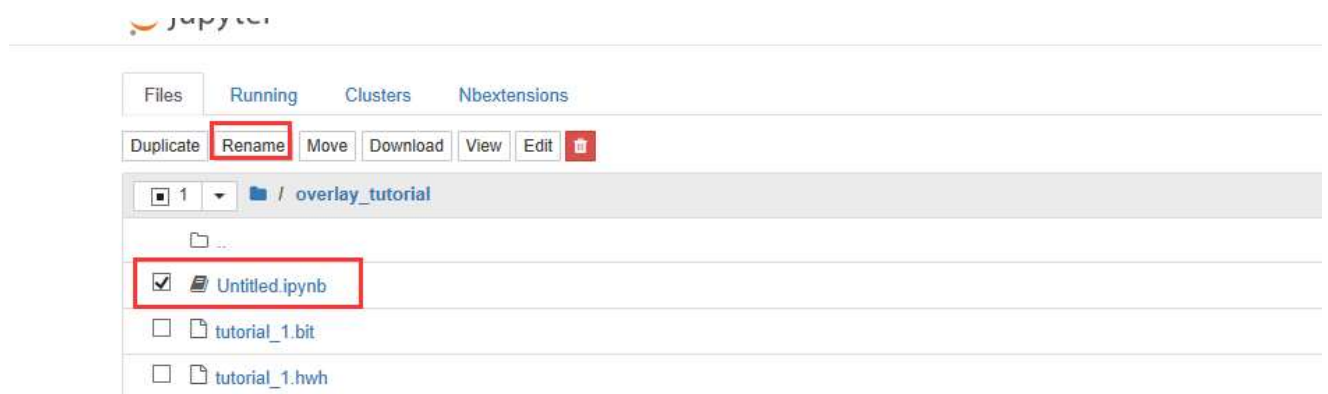
```
In [7]: add_ip.write(0x10, 4)
add_ip.write(0x18, 5)
add_ip.read(0x20)
```

10. Due to the detailed annotations in the notebook, this part of the python program will not be explained

11. Finally, don't forget to click Save



12. For the sake of memorization, I changed the botebook to "tutorial_1.ipynb"



13. Access \\192.168.x.xxx\xilinx\jupyter_notebooks\overlay_tutorial through samba, you can find the files we uploaded and created.

含有DMA操作自定义Overlays

目录

- DMA如何操作

In the previous example, there are only register read and write operations, and DMA is often used in large-scale high-speed data interactions, so overlays containing DMA are very important.

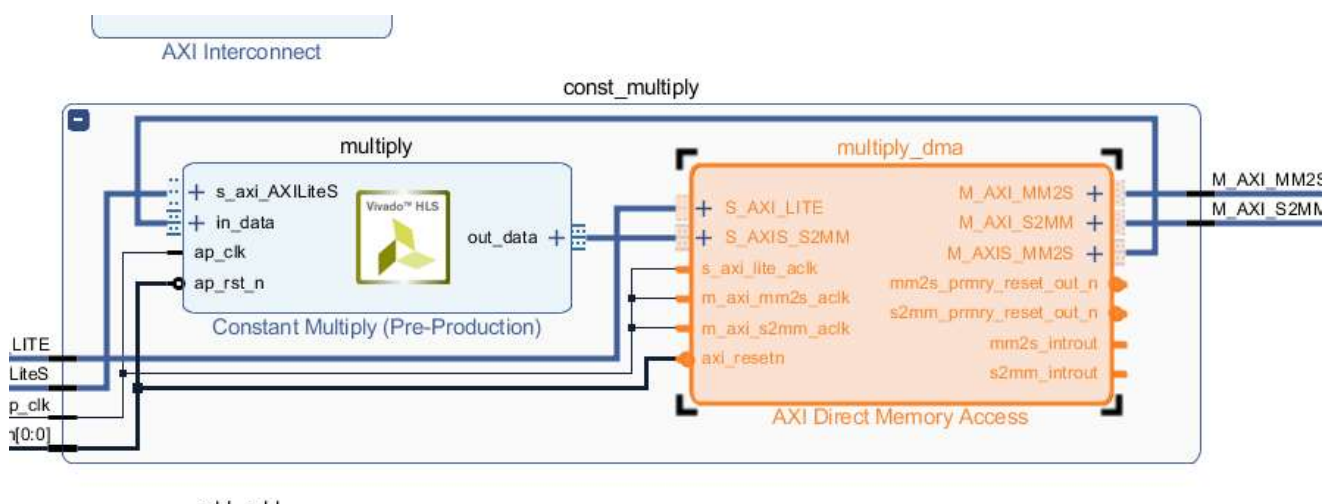
This experiment is also a stream multiplier written based on HLS, that is, one array is multiplied by one constant to get another array

```
#include "ap_axi_sdata.h"
#include "ap_int.h"

typedef ap_axiu<32,1,1,1> stream_type;

void mult_constant(stream_type* in_data, stream_type* out_data, ap_int<32> constant) {
    #pragma HLS INTERFACE s_axilite register port=constant
    #pragma HLS INTERFACE ap_ctrl_none port=return
    #pragma HLS INTERFACE axis port=in_data
    #pragma HLS INTERFACE axis port=out_data
    out_data->data = in_data->data * constant;
    out_data->dest = in_data->dest;
    out_data->id = in_data->id;
    out_data->keep = in_data->keep;
    out_data->last = in_data->last;
    out_data->strb = in_data->strb;
    out_data->user = in_data->user;
}
```

tutorial_2 is the vivado project of this experiment, where we can see that multiply and multiply_dma are combined into a const_multiply, which is equivalent to a submodule that needs to be divided into hierarchies when referenced in PYNQ



Just like in the code, `dma = overlay.const_multiply.multiply_dma`, first referencing `const_multiply` and then referencing `multiply_dma`

```
: import pynq.lib.dma

overlay = Overlay('./tutorial_2.bit')

dma = overlay.const_multiply.multiply_dma
multiply = overlay.const_multiply.multiply
```

How DMA works:

Combine the code to illustrate the DMA operation

1. First, import the Xlnk library, which contains Xilinx's DMA management interface.
2. Use `cma_array` to allocate DMA buffers
3. Use `sendchannel.transfer` to initiate a DMA send operation
4. Use `recvchannel.transfer` to initiate a DMA acceptance operation
5. `sendchannel.wait()` wait for the sending to complete
6. `recvchannel.wait()` wait for the receive to complete

```
from pynq import Xlnk
import numpy as np

xlnk = Xlnk()
in_buffer = xlnk.cma_array(shape=(5,), dtype=np.uint32)
out_buffer = xlnk.cma_array(shape=(5,), dtype=np.uint32)

for i in range(5):
    in_buffer[i] = i

multiply.constant = 3
dma.sendchannel.transfer(in_buffer)
dma.recvchannel.transfer(out_buffer)
dma.sendchannel.wait()
dma.recvchannel.wait()

out_buffer
```

ADC采集并显示波形

目录

- ADC硬件
- Vivado工程
- Notebook文件

In the previous experiment, we learned to use PYNQ to operate dma, but it is not very intuitive, so this experiment uses a more intuitive example to illustrate the use of dma by collecting ADC data and displaying waveforms.

ADC hardware

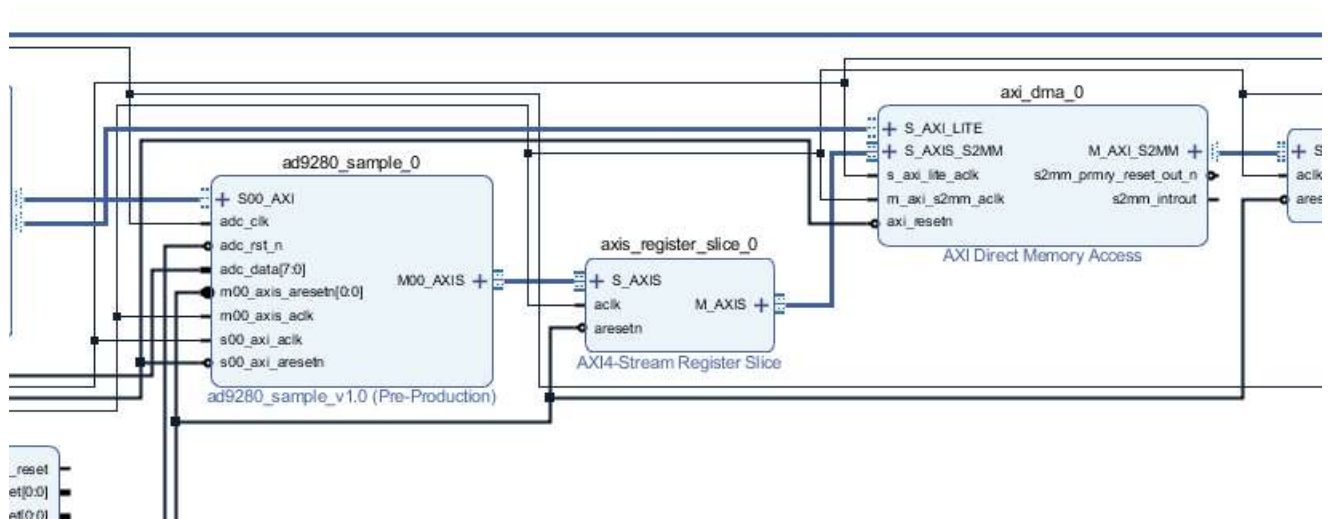
The module used in the experiment is AN108, one 32M sampling rate ADC, and one 125M sampling rate DAC, and the ADC channel is used in the experiment. We have already used the AN108 module in the SDK routine, so we won't explain it here.



Vivado Project

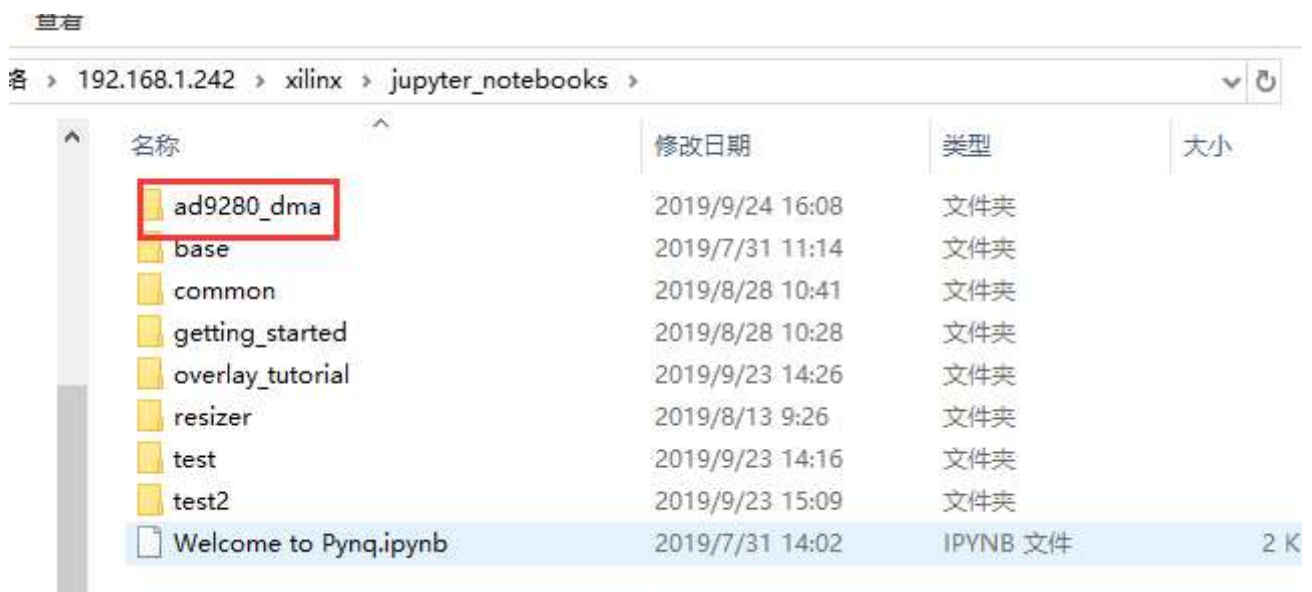
The Vivado project comes from the ad9280_dma_hdmi of course_s2, here the HDMI display part is removed, the DMA disconnection is removed, and this Vivado project will not be

explained here.



Notebook file

1. Access `\\192.168.x.xxx\xilinx\jupyter_notebooks` through Samba and create a new folder `ad9280_dma` store bit files, hwh files, and jupyter_notebook files



2. Copy the `design_1_wrapper.bit` file to `\\192.168.x.xxx\xilinx\jupyter_notebooks\ad9280_dma` and rename it to `ad9280_dma.bit`

test > PYNQ > ax7010 > ad9280_dma > ad9280_dma.runs > impl_1 >		
名称	修改日期	类型
.Xil	2019/9/24 15:45	文件夹
.init_design.begin.rst	2019/9/24 15:43	RST 文件
.init_design.end.rst	2019/9/24 15:44	RST 文件
.opt_design.begin.rst	2019/9/24 15:44	RST 文件
.opt_design.end.rst	2019/9/24 15:44	RST 文件
.place_design.begin.rst	2019/9/24 15:44	RST 文件
.place_design.end.rst	2019/9/24 15:44	RST 文件
.route_design.begin.rst	2019/9/24 15:44	RST 文件
.route_design.end.rst	2019/9/24 15:45	RST 文件
.vivado.begin.rst	2019/9/24 15:43	RST 文件
.vivado.end.rst	2019/9/24 15:45	RST 文件
.Vivado_Implementation.queue.rst	2019/9/24 15:40	RST 文件
.write_bitstream.begin.rst	2019/9/24 15:45	RST 文件
.write_bitstream.end.rst	2019/9/24 15:45	RST 文件
design_1_wrapper.bit	2019/9/24 15:45	BIT 文件

- Copy design_1.hwh to \\192.168.x.xxx\xilinx\jupyter_notebooks\ad9280_dma and rename it to ad9280_dma.hwh

ad9280_dma > ad9280_dma.srscs > sources_1 > bd > design_1 > hw_handoff			
名称	修改日期	类型	大
design_1.hwh	2019/9/24 15:40	HWH 文件	
design_1_bd.tcl	2019/9/24 15:40	TCL 文件	

- If you don't want to edit the notebook yourself, copy the ad9280_dma.ipynb file to \\192.168.x.xxx\xilinx\jupyter_notebooks\ad9280_dma

192.168.1.242 > xilinx > jupyter_notebooks > ad9280_dma			
名称	修改日期	类型	大小
ad9280_dma.bit	2019/9/24 15:45	BIT 文件	2,036 KB
ad9280_dma.hwh	2019/9/24 15:40	HWH 文件	281 KB
ad9280_dma.ipynb	2019/9/24 16:08	IPYNB 文件	32 KB

- Browser login to open the notebook

Files Running Clusters Nbextensions

Select items to perform actions on them.

Upload New ↕

☐ 0	/ ad9280_dma	Name ↓	Last Modified
☐	..		几秒前
☐	ad9280_dma.ipynb		20 分钟前
☐	ad9280_dma.bit		43 分钟前
☐	ad9280_dma.hwh		1 小时前

6. Wait for the kernel start to finish before running



Kernel starting, please wait...

Trusted

Python 3 ●

File Edit View Insert Cell Kernel Widgets Help

7. 100Khz sine wave, running result

```
In [14]: #获取AD采集到的数据并显示
dma.recvchannel.transfer adc_buffer)
adc_sample.write(0x04, 1024) #sample 1024 points
adc_sample.write(0x00, 0x01) #start sample
dma.recvchannel.wait()
#画图
fig1 = plt.figure()
ax1 = fig1.gca()
plt.plot(adc_buffer)
plt.cla
```

```
Out[14]: <function matplotlib.pyplot.cla>
```

